Extended Relational Model

The Extended Relational Model is an enhancement of the traditional relational model designed to support complex and real-world data more effectively.

Extension over the Traditional Relational Model:
The traditional relational model supports only simple, atomic data values. The extended relational model overcomes this limitation by allowing complex data types and advanced data modeling features. It provides better support for modern applications such as multimedia systems, CAD (Computer-Aided Design), and scientific databases.

Key Features:

1. **Complex Data Types**
   It supports non-atomic attributes such as arrays, lists, and nested structures.
   Example: A Student table may store multiple phone numbers for a single student.
2. **User-Defined Types (UDTs)**
   Users can define their own data types to represent real-world objects.
   Example: A Address type containing street, city, and zip code.
3. **Inheritance**
   Tables can inherit attributes from other tables, similar to object-oriented programming.
   Example: A GraduateStudent table inheriting from a Student table.
4. **Object Identity**
   Each object can have a unique identity independent of its values.
   Example: Two employee records with the same details but different identities.
5. **Methods (Functions)**
   Data and operations can be stored together.
   Example: A method to calculate the total salary of an employee.

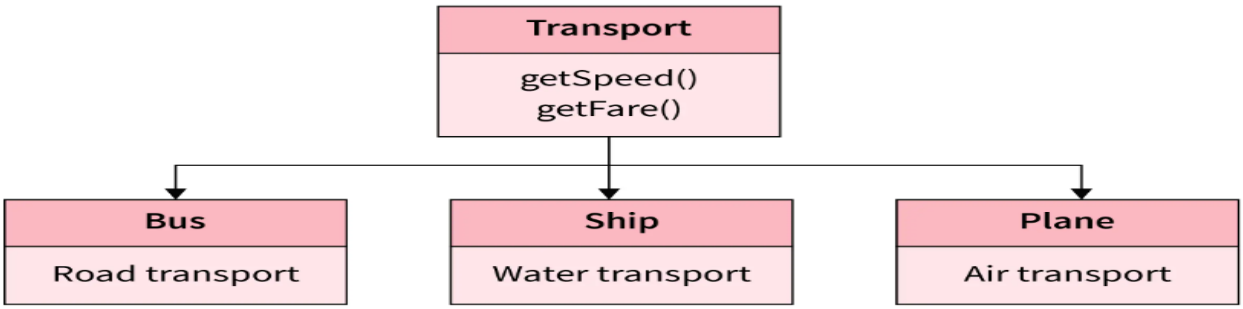
Object Oriented Model

The Object-Oriented Model in DBMS or OODM is the data model where data is stored in the form of objects. This model is used to represent real-world entities. The data and data relationship are stored together in a single entity known as an object in the Object Oriented Model. The Object-Oriented Database Management System is built on top of Object Oriented Model.

We can use the Object Oriented Model in DBMS to store real-world entities. Here, we can store pictures, audio, video, and other types of data, which was previously impossible to store with the relational approach (Even though we can store video and audio in the relational database, it is generally not recommended).

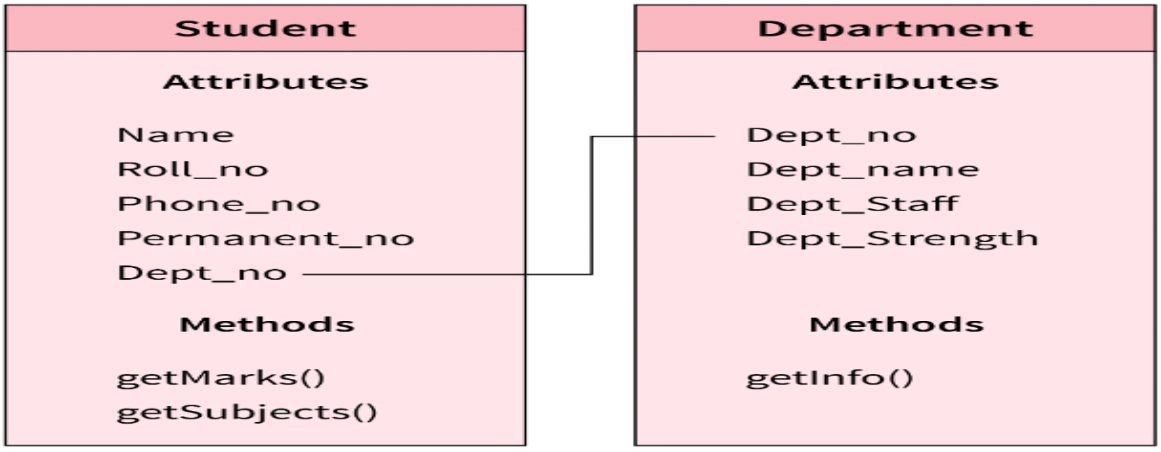
Example

Now let us consider an example given below.



- Here Transport, Bus, Ship, and Plane are objects.
- Bus has Road Transport as the attribute.
- Ship has Water Transport as the attribute.
- Plane has Air Transport as the attribute.
- The Transport object is the base object and the Bus, Ship, and Plane objects derive from it.

Take a look at another example-



As you can see, here Student and Department are two different objects. Each one of them has its attributes and methods. They are linked by a common attribute

Dept_no which establishes a relationship between objects.

Components of Object-Oriented Data Model

Components of the Object-Oriented Data Model namely objects, classes, object attributes, class hierarchy, etc., are explained as follows-

Objects: Objects represent real-world entities.

Example: A specific book in the library.

Classes: Classes are blueprints for creating objects.

Example: A "Car" class defines attributes (color, model) and methods (start(), stop()), and different car objects (red car, blue car) can be created from it.

Attributes: Attributes are the properties of an object.

Example: The "Book" class might have attributes like title, author, and ISBN.

Methods: Methods are the actions or behaviors an object can perform.

Example: The "Book" class might have methods like borrow, return.

Inheritance- A mechanism by which one class (called a subclass) can inherit attributes and methods from another class (called a superclass). This allows for code reuse and the creation of more specific classes based on general ones. For example, a "SportsCar" class might inherit from a "Car" class but add additional attributes like top speed.

Distributed Database Definition

A distributed database is a type of database where data is stored across multiple computers or locations, but it all works together as one system.

Imagine you have a big book, but instead of keeping it in one place, you split it into smaller parts and keep those parts in different locations. When you need to read the book, you can still access all the parts as if they were in one place.

A distributed database represents multiple interconnected databases spread out across several sites connected by a network. Since the databases are all connected, they appear as a single database to the users.

Distributed Database Features

Some general features of distributed databases are:

- **Location independency** - Data is physically stored at multiple sites and managed by an independent DDBMS.
- **Distributed Query Processing**:
  It means getting answers to queries when data is stored in different places. The system breaks a big query into small parts and processes them at different locations.
- **Distributed Transaction Management**:
  It makes sure all database operations work correctly across different places. It ensures that transactions are completed properly, even when many users are working at the same time or when failures happen.
- **Seamless integration** - Databases in a collection usually represent a single logical database, and they are interconnected.
- **Network linking** - All databases in a collection are linked by a network and communicate with each other.
- **Transaction processing** - Transaction processing is an atomic process that is either entirely executed or not at all.

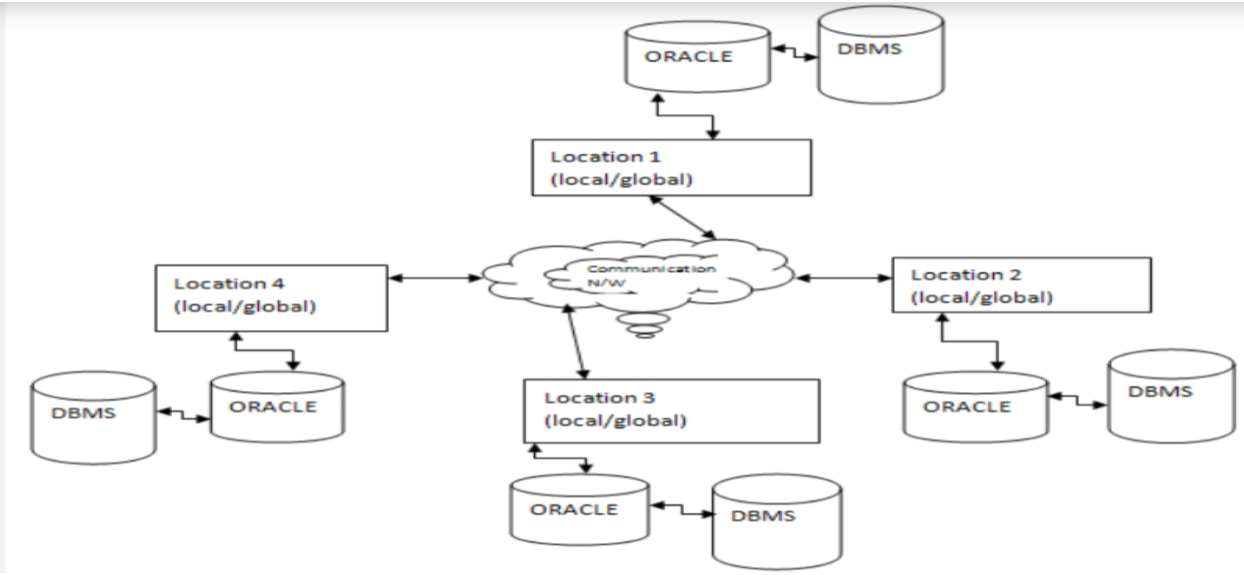
Distributed Database Types

There are two types of distributed databases:

- **Homogenous**
- **Heterogenous**
    1. **Homogenous**

- **Same DBMS:** All databases use the same database management system (DBMS) software.
- **Uniform Schema:** The schema, or the structure of the data, is the same across all databases.

- **Simpler Integration:** Since all nodes are uniform, data integration and query processing are straightforward.



Advantages:

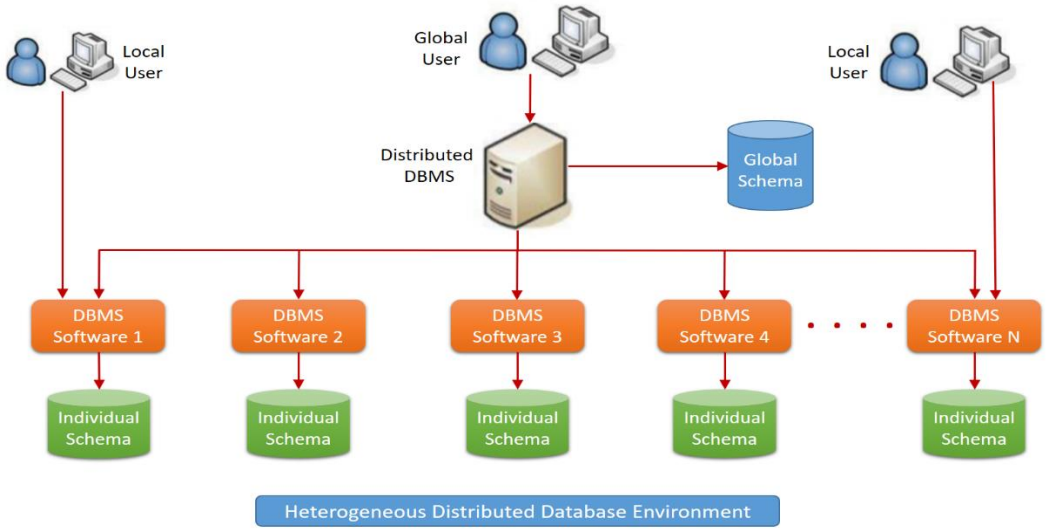
- Easier to manage and maintain because of uniformity.
- Simplified replication and data distribution.
- Easier query optimization since the DBMS is consistent across the network.

Disadvantages:

- Limited flexibility as all nodes must use the same software.
- Potential issues if the DBMS lacks features needed for certain applications.

2. Heterogeneous Distributed Databases

- **Different DBMS:** The system integrates different types of DBMS software.
- **Diverse Schema:** Schemas may differ across the databases.
- **Complex Integration:** Requires middleware or translation layers to handle differences in DBMS and schemas.



Advantages:

- Flexibility to use the most suitable DBMS for different applications.
- Can integrate existing databases without needing to convert them to a single DBMS.

Disadvantages:

- Complex to manage due to the heterogeneity.
- Performance issues might arise from the need to translate queries and data between different systems.
- More challenging to implement global transactions and ensure consistency.

Distributed Database Storage

Distributed database storage is managed in two ways:

- **Replication**
- **Fragmentation**

Replication

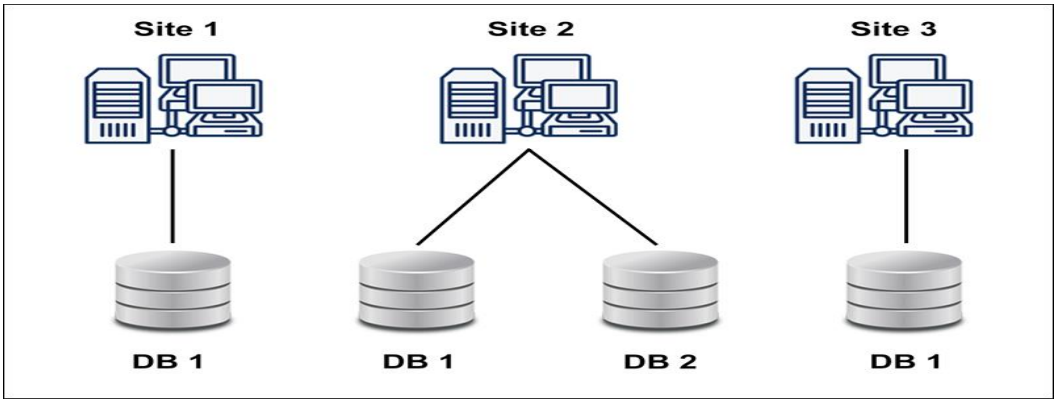
In database replication, the systems store **copies of data on different sites**. If an entire database is available on multiple sites, it is a fully redundant database.

The advantage of database replication is that it **increases data availability o**n different sites and allows for parallel query requests to be processed.

Note: site= computer in distributed system

However, database replication means that data requires constant updates and synchronization with other sites to maintain an exact database copy. Any changes made on one site must be recorded on other sites, or else inconsistencies occur.

Constant updates cause a lot of server overhead and complicate concurrency control, as a lot of concurrent queries must be checked in all available sites.



Fragmentation

When it comes to fragmentation of distributed database storage, the relations are fragmented, which means they are **split into smaller parts**. Each of the fragments is stored on a different site, where it is required.

The prerequisite for fragmentation is to make sure that the fragments can later be reconstructed into the original relation without losing data.

The advantage of fragmentation is that there are **no data copies**, which prevents data inconsistency.

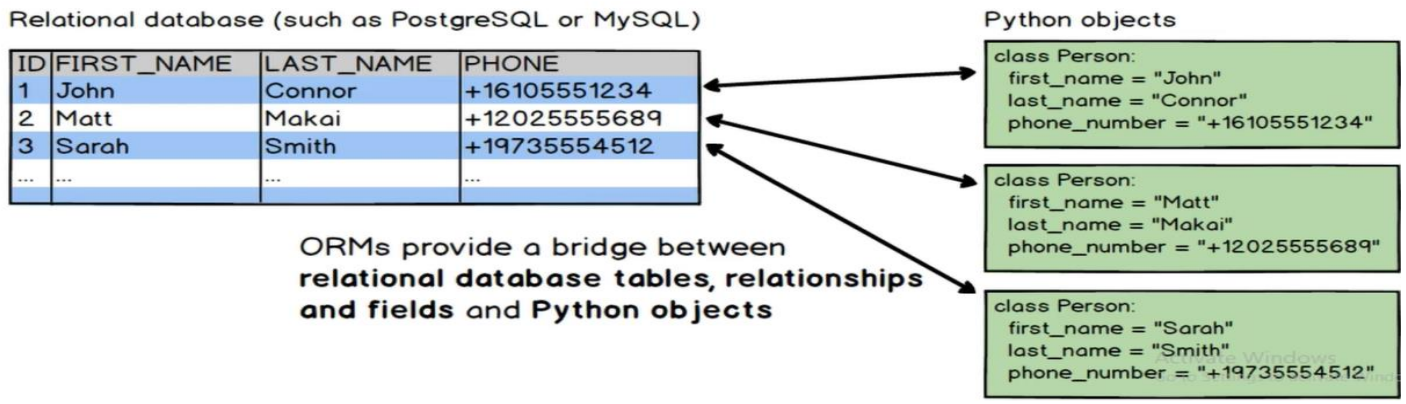
There are two types of fragmentation:

- **Horizontal fragmentation** - The relation schema is fragmented into groups of rows, and each group (tuple) is assigned to one fragment.
- **Vertical fragmentation** - In **vertical fragmentation**, you **split the table by columns**. Each fragment contains **some of the columns**, but they all **include the same primary key or candidate key** (to identify rows uniquely). This ensures you can **join them back without losing data**.

Object-relational Data Model

An Object relational model is a combination of a Object oriented database model and a Relational database model. So, it supports objects, classes, inheritance etc. just like Object Oriented models and has support for data types, tabular structures etc. like Relational data model.

One of the major goals of Object relational data model is to close the gap between relational databases and the object oriented.



- ORDBMS (Object-Relational Database Management Systems) extend traditional relational databases by adding features from object-oriented programming.
- They act as a bridge between relational databases and object-oriented systems.
- ORDBMS were developed to handle complex data types like audio, video, and images, which ordinary relational databases could not manage well.
- Their development became necessary because object-oriented programming languages were widely used, and there was a mismatch between these languages and traditional relational DBMSs.
- An Object–Relational Mapper (ORM) automatically maps objects in a programming language (like Java, Python, or C#) to tables in a relational database**,** and vice versa.

History of Object Relational Data Model

Both Relational data models and Object oriented data models are very useful. But it was felt that they both were lacking in some characteristics and so work was started to build a model that was a combination of them both. Hence, Object relational data model was created as a result of research that was carried out in the 1990's.

Advantages of ORM

1. **Simplifies Database Interactions**: Developers can use the programming language they are familiar with to interact with the database, reducing the need for complex SQL queries.
2. **Reduces Errors**: ORM automates many database tasks, reducing the risk of errors that can occur with manual SQL queries.
3. **Improves Productivity**: Developers can focus more on business logic rather than database management, speeding up development.

Disadvantages of ORM

1. **Complexity:** The ORM model is more complex than purely relational or purely object-oriented models. Mapping objects to relational tables (object-relational mapping) requires additional logic and tools.
2. **Limited Flexibility with Complex Queries:** Complex SQL queries involving multiple joins, subqueries, or performance tuning are harder to express using ORM tools.
3. **Learning Curve**: Developers need to learn how to use the ORM tool effectively, which can take some time.

Question

1. Explain the Extended Relational Model. How does it enhance the traditional relational model? Discuss its key features with suitable examples.
2. Describe the Object-Oriented Database Model. Explain its main concepts such as objects, classes, inheritance, and encapsulation.

3. What is an Object-Relational Database Model? Explain how it bridges the gap between relational and object-oriented models. Mention its advantages.
4. Define a Distributed Database. Explain its characteristics and advantages over centralized databases.
5. Compare the Extended Relational Model, Object-Oriented Model, and Object-Relational Model.